



Machine Learning

A) BAYES CLASSIFIER

B) NONPARAMETRIC CLASSIFIERS

Cigdem Beyan

A. Y. 2025/2026

INTRODUCTION

- A fundamental **statistical approach** for **classification**.
- Hypothesis:
 - The decision problem is cast in **probabilistic terms**,
 - All relevant **probabilities are known**.
- Goal:
 - Discriminate the different decision rules using the probabilities and the associated costs
 - Estimating the **joint probability $p(x, y)$** from the training data set.

AN EASY EXAMPLE

- Let y is what we want to classify and it is to be probabilistically described
- There are **two classes** y_1 and y_2 which correspond to two possible states:
 - a) $P(y = y_1) = 0.7$
 - b) $P(y = y_2) = 0.3$

→ a-priori or prior probability
- There are no measurements or observations available to help make a decision about which class y belongs to.
- Decision rule:
 - Decide y_1 if $P(y_1) > P(y_2)$; otherwise decide y_2
 - Given the probabilities (70% for y_1 and 30% for y_2), the decision rule would lead to classifying y as y_1 since 0.7 is greater than 0.3.

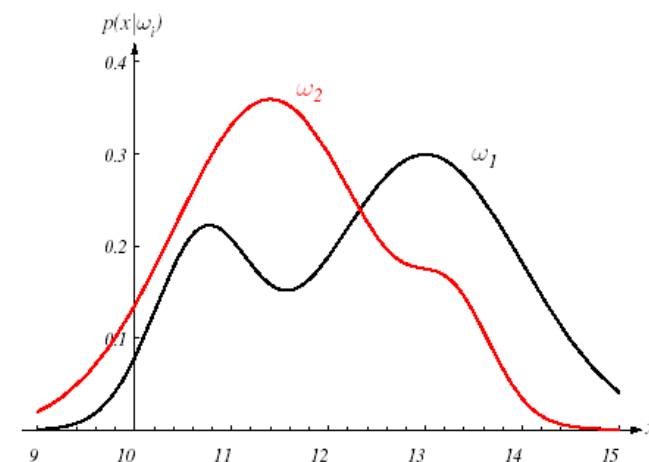
ANOTHER EXAMPLE

- Having the previous hypothesis and additionally **a single measurement $\mathbf{x}$** , which is a random variable that depends on the class y_j , we can get:

$$p(x|y_j)_{j=1,2} = \textbf{Likelihood or class-conditional probability density function}$$

i.e. *the probability of having the measurement x knowing that y is y_j*

- When we fix the measurement $\mathbf{x}$, the higher $p(x|y_j)$ is, the more likely y_j is the correct state.



BAYES FORMULA

- Assuming known $\mathbf{P(y_j)}$ and $\mathbf{p(x|y_j)}$, the decision of ω becomes, for Bayes

$$p(y_j, x) = P(y_j|x)p(x) = p(x|y_j)P(y_j)$$

that is

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)} \propto p(x|y_j)P(y_j)$$

where:

$$P(y_j) = \mathbf{Prior}$$

$$p(x|y_j) = \mathbf{Likelihood}$$

$$P(y_j|x) = \mathbf{Posterior}$$

$$p(x) = \sum_{j=1}^J p(x|y_j) P(y_j) = \mathbf{Evidence}$$

BAYES DECISION RULE

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)} \quad \longleftrightarrow \quad \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

- The posterior or ***a-posteriori probability*** is the probability that y is y_j given the observation x .
- The most important factor is the product $\text{likelihood} \times \text{prior}$, the evidence $p(x)$ is simply a scale factor, which ensures that

$$\sum_j P(y_j|x) = 1$$

- From the formula of Bayes derives the **Bayes' decision rule**:
Decide y_1 if $P(y_1|x) > P(y_2|x)$, and y_2 otherwise

NAÏVE BAYES CLASSIFIER

- A probabilistic classification algorithm based on **Bayes' theorem**.
- Assumes **independence** among features.
- Calculates the **posterior probability** using

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)}$$

- Assumes features $x_1, x_2, \dots, x_n$ are independent given the class label ω :

$$P(x | y) = P(x_1 | y) \cdot P(x_2 | y) \cdots P(x_n | y) \qquad P(x | y) = \prod_{i=1}^n P(x_i | y)$$

- Probability of each class is determined from training data.



EXAMPLE

We want to classify an email as either:

- **Class 1:** *Spam*
- **Class 2:** *Not spam (Ham)*

We'll use a few features (words) that appear in the email.

Email	Contains “Free”?	Contains “Win”?	Class
E1	Yes	Yes	Spam
E2	Yes	No	Spam
E3	No	Yes	Spam
E4	No	No	Ham
E5	No	No	Ham

EXAMPLE

Step 1: Compute class priors $P(\text{Spam}) = \frac{3}{5}, \quad P(\text{Ham}) = \frac{2}{5}$

Step 2: For each feature (word) and class, we estimate how often the word appears when the email belongs to that class.

$$P(\text{Free} = \text{Yes} \mid \text{Spam}) = \frac{2}{3}, \quad P(\text{Free} = \text{Yes} \mid \text{Ham}) = \frac{0}{2} = 0$$

$$P(\text{Win} = \text{Yes} \mid \text{Spam}) = \frac{2}{3}, \quad P(\text{Win} = \text{Yes} \mid \text{Ham}) = \frac{0}{2} = 0$$

EXAMPLE

Step 3 should be calculation of evidence but in practice, for **classification**, $P(x)$ is the same for all classes — it's just a normalization constant. Therefore, we can skip its calculation to estimate the posterior.

Step 3: Classify a new email (validation/testing)

New email: contains “**Free**”, does **not** contain “**Win**”.

We compute (ignoring the denominator (evidence) since it's the same for both classes):

$$\begin{aligned} P(\text{Spam} \mid \text{Free, no Win}) &\propto P(\text{Spam}) \cdot P(\text{Free}|\text{Spam}) \cdot P(\text{no Win}|\text{Spam}) \\ &= \frac{3}{5} \times \frac{2}{3} \times \left(1 - \frac{2}{3}\right) = \frac{3}{5} \times \frac{2}{3} \times \frac{1}{3} \end{aligned}$$

$$\begin{aligned} P(\text{Ham} \mid \text{Free, no Win}) &\propto P(\text{Ham}) \cdot P(\text{Free}|\text{Ham}) \cdot P(\text{no Win}|\text{Ham}) \\ &= \frac{2}{5} \times 0 \times 1 = 0 \end{aligned}$$

The model predicts **Spam**, because its probability is greater.

NAÏVE BAYES CLASSIFIER

- Classify an observation x to class y_j if:

$$P(y_j|x) = \frac{p(x|y_j)P(y_j)}{p(x)} \quad \text{is maximized}$$

- **Advantages:**

- Fast to train and predict, suitable for large datasets.
- Works well with high-dimensional data.

- **Limitations:**

- Independence assumption may not hold in real-world scenarios, leading to suboptimal performance.
- Zero probability problem: requires techniques for unseen feature-class combinations.

PROBLEMS

- To create an optimal classifier that uses the Bayesian decision rule, you need to know:
 - **Prior probabilities** $P(y_i)$
 - **Class-conditional densities (likelihood)** $p(\mathbf{x} | y_i)$
- The performance of a classifier strongly depends on the **goodness** of these components.

BUT PRACTICALLY ALL THIS INFORMATION IS NEVER AVAILABLE!!



PROBLEMS

- More often we only have:
 - A vague knowledge of the problem, from which to extract vague **a-priori probabilities**.
 - Some particularly representative patterns, training data, used to train the classifier (**often too few or sparse!**)
- Estimating a-priori probabilities is usually not particularly difficult.
- Estimating conditional densities (likelihood) is **more complex** → needs approximations.



PROBLEMS

- Estimating conditional densities (likelihood) is **more complex** → needs approximations.
 - Data can have many features → exponential combinations (high-dimensional problem)
 - Some feature combinations may never appear in training data → zero probabilities
 - Features may be continuous, requiring parametric assumptions (e.g., Gaussian)

SOLUTION (ONE SOLUTION OUT OF MANY)

- We approximate it by **fitting a parametric model** (e.g., Gaussian) to the training data and estimate its parameters using e.g., **Maximum Likelihood Estimation (MLE)**.
- It is a standard way to estimate parameters associated to the likelihood from training data.

$$\theta_j = (\mu_j, \Sigma_j)$$

$$P(x | y_j) \approx N(\mu_j, \Sigma_j)$$

MLE chooses the parameter values that make the observed data **most probable**.

MLE IN NAÏVE BAYES (DISCRETE CASE)

Suppose you're training a Naïve Bayes classifier with **categorical** features.
For each class y , you estimate:

- the **prior** $P(y)$ and the **likelihoods** $P(x_i | y)$

(a) Prior via MLE
$$P(y) = \frac{\text{number of samples with class } y}{\text{total number of samples}}$$

(b) Likelihood via MLE

If x_i is a categorical feature (e.g., word):
$$P(x_i = v_k | y) = \frac{\text{count of } (x_i = v_k, y)}{\text{count of } y}$$

This is the **maximum likelihood estimate** of the conditional probability $P(x_i = v_k | y)$. You're simply estimating probabilities by **relative frequencies**: the parameter values that **maximise** the likelihood of your training data.

CONTINUOUS FEATURES: GAUSSIAN MLE

If features are continuous and you assume a Gaussian likelihood:

$$P(x_i | y) = \mathcal{N}(x_i; \mu_{iy}, \sigma_{iy}^2) \quad \Rightarrow \quad P(x_i | y) = \frac{1}{\sqrt{2\pi\sigma_{iy}^2}} \exp\left(-\frac{(x_i - \mu_{iy})^2}{2\sigma_{iy}^2}\right)$$

Then MLE gives:

$$\hat{\mu}_{iy} = \frac{1}{N_y} \sum_{j:y_j=y} x_{ij}, \quad \hat{\sigma}_{iy}^2 = \frac{1}{N_y} \sum_{j:y_j=y} (x_{ij} - \hat{\mu}_{iy})^2$$

These are just the **sample mean and variance** of that feature within each class.

EXAMPLE

Fruit	Weight (g)
Apple	150
Apple	160
Apple	155
Orange	180
Orange	190
Orange	185

(1) Estimate Gaussian parameters (MLE)

For Apples:

$$\mu_{\text{Apple}} = \frac{150 + 160 + 155}{3} = 155$$

$$\sigma_{\text{Apple}}^2 = \frac{(150 - 155)^2 + (160 - 155)^2 + (155 - 155)^2}{3} = \frac{25 + 25 + 0}{3} \approx 16.67$$

For Oranges:

$$\mu_{\text{Orange}} = \frac{180 + 190 + 185}{3} = 185$$

$$\sigma_{\text{Orange}}^2 = \frac{(180 - 185)^2 + (190 - 185)^2 + (185 - 185)^2}{3} = \frac{25 + 25 + 0}{3} \approx 16.67$$

EXAMPLE

(2) Compute the likelihood for a new sample

New fruit weight: $x = 158\text{g}$.

Gaussian likelihood formula: $P(x | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x - \mu_y)^2}{2\sigma_y^2}\right)$

- Apple:

$$\begin{aligned} P(158 | \text{Apple}) &= \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 155)^2}{2 \cdot 16.67}\right) \\ &\approx 0.097 \cdot \exp(-0.27) \approx 0.072 \end{aligned}$$

- Orange:

$$\begin{aligned} P(158 | \text{Orange}) &= \frac{1}{\sqrt{2\pi \cdot 16.67}} \exp\left(-\frac{(158 - 185)^2}{2 \cdot 16.67}\right) \\ &\approx 0.097 \cdot \exp(-22.5) \approx 1.6 \times 10^{-11} \end{aligned}$$

EXAMPLE

(3) Multiply by class prior (assume equal)

$$P(\text{Apple}) = P(\text{Orange}) = 0.5$$

Apple posterior: $0.072 \cdot 0.5 \approx 0.036$

Orange posterior: $1.6 \times 10^{-11} \cdot 0.5 \approx 8 \times 10^{-12}$

Prediction: Apple, because posterior probability is much higher.

OTHER SOLUTIONS

Non-parametric methods

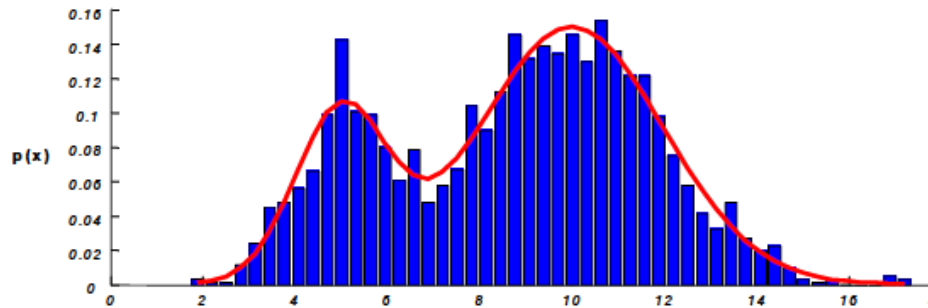
- Given a set of examples, model the density function of the data without making any assumptions about the **form of the distribution** (e.g., Gaussian distribution)
 - Estimate the probability density function **directly from the samples**.
- The simplest form of non-parametric density estimation is the **histogram**.

HISTOGRAM

- Dividing the sample space into a number of **bins** and approximate the density at the **center of each bin** by the **fraction** of points in the training data that fall into the corresponding bin,

$$p_H(x) = \frac{1}{N} \frac{[\# \text{ of } x^{(k)} \text{ in same bin as } x]}{[\text{width of bin}]}$$

- The histogram requires two “parameters” to be defined: **bin width** and **starting position** of the first bin.





HISTOGRAM

- Several drawbacks
 - The density estimate depends on the **starting position of the bins**
 - For multivariate data, the density estimate is also affected by the orientation of the bins
 - The **discontinuities** of the estimate are not due to the underlying density; they are only an artifact of the chosen bin locations
 - These discontinuities make it very difficult to grasp the **structure** of the data
 - A much more serious problem is the **curse of dimensionality**, since the number of bins grows exponentially with the number of dimensions
 - In high dimensions we would require a very large number of examples or else most of the **bins** would be **empty**.



- **Curse of dimensionality** refers to the challenges and issues that arise when working with high-dimensional data in machine learning.
 - As the number of features or dimensions in a dataset increases, the amount of data required to effectively cover the feature space also increases exponentially.
 - To mitigate the curse of dimensionality, various techniques such as **feature selection**, **dimensionality reduction** (e.g., principal component analysis), and **regularization methods** are employed to reduce the dimensionality of the data while preserving its important characteristics.

NONPARAMETRIC TECHNIQUES

Before we proceed any further let us return to the basic definition of probability to get a solid idea of what we are trying to accomplish

- The probability that a vector $\mathbf{x}$, drawn from a distribution $P(\mathbf{x})$, will fall in a region $\mathcal{R}$ of the sample space is

$$P = \int_{\mathbf{x}' \in \mathcal{R}} P(\mathbf{x}') d\mathbf{x}'$$

where

$\mathbf{x}$ is a particular vector we are asking about (“drawn from the distribution”).

$\mathbf{x}'$ is a dummy variable used to sum/integrate over all vectors in R .

NONPARAMETRIC TECHNIQUES

- Suppose now that N vectors $\{x(1), x(2), \dots, x(N)\}$ are drawn ***independently and identically distributed (i.i.d.)*** from the distribution. The probability that $\mathbf{k}$ of these $\mathbf{N}$ vectors fall in $\mathfrak{R}$ is given by the binomial distribution

$$P(\mathbf{k}) = \binom{N}{k} P^k (1-P)^{N-k} \quad \text{where} \quad \binom{N}{k} = \frac{N!}{k! \cdot (N-k)!}$$

and the expected value for $\mathbf{k}$ is **$E[\mathbf{k}] = NP$**

Recall: The expectation is the average value we would get if we repeated drawing samples many times.

NONPARAMETRIC TECHNIQUES

- The mean and variance of the ratio k/N are

$$E\left[\frac{k}{N}\right] = P \quad \text{and} \quad \text{Var}\left[\frac{k}{N}\right] = E\left[\left(\frac{k}{N} - P\right)^2\right] = \frac{P(1-P)}{N}$$

- Therefore, as $N \rightarrow \infty$, the distribution becomes **sharper**, the variance gets **smaller**, thus we can obtain a **better estimate** of the probability P from the mean fraction of the points that fall within $\mathfrak{R}$

$$P \cong \frac{k}{N}$$

- On the other hand, if we assume that $\mathfrak{R}$ is **so small** that $P(x)$ **does not vary** appreciably within it, then

$$\int_{\mathfrak{R}} P(x') dx' \cong P(x)V$$

where V is the volume enclosed by region $\mathfrak{R}$.

NONPARAMETRIC TECHNIQUES

- Merging with the previous result, we obtain

$$\left. \begin{array}{l} P = \int_{\mathfrak{R}} p(x') dx' \cong p(x)V \\ P \cong \frac{k}{N} \end{array} \right\} \Rightarrow p(x) \cong \frac{k}{NV}$$

- This estimate becomes more accurate as we increase the number of sample points N and shrink the volume V

NONPARAMETRIC TECHNIQUES

- In practice the value of N (the total number of examples) is fixed
 - In order to improve the accuracy of the estimate $P(x)$ we could let V to approach **zero**, but then the region $\mathfrak{R}$ would become so **small** that it would enclose no examples.
- This means that in practice, we will have to find **a compromise value of the volume V** ,
 - Large enough to include enough examples within $\mathfrak{R}$
 - Small enough to support the assumption that $P(x)$ is constant within $\mathfrak{R}$.

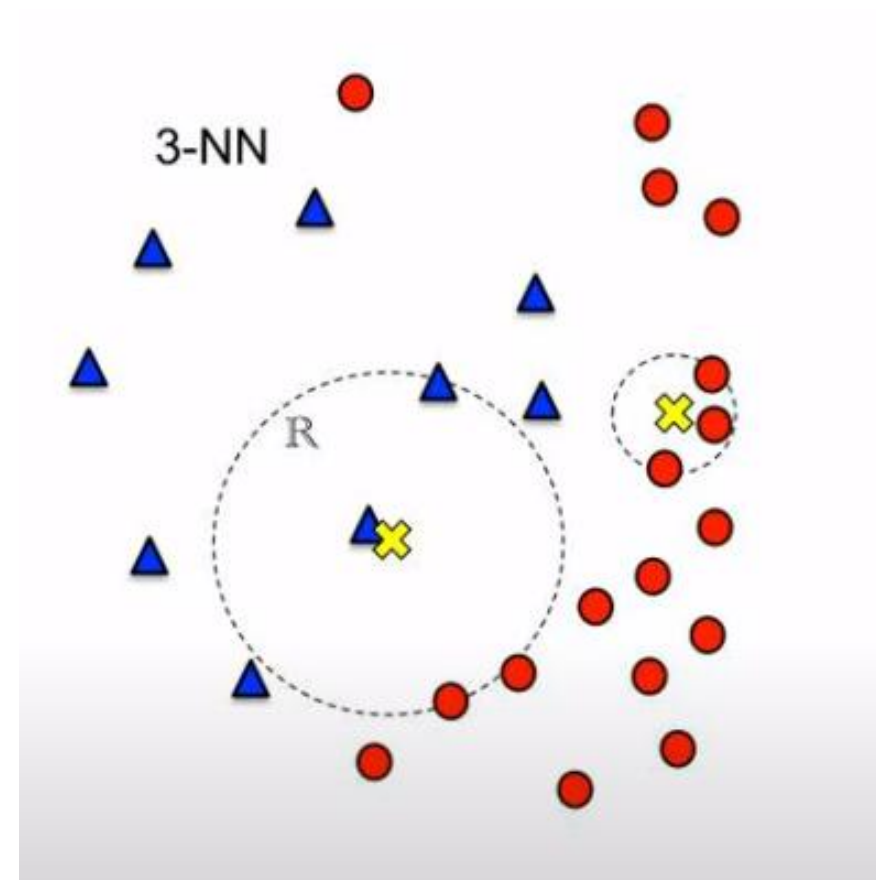
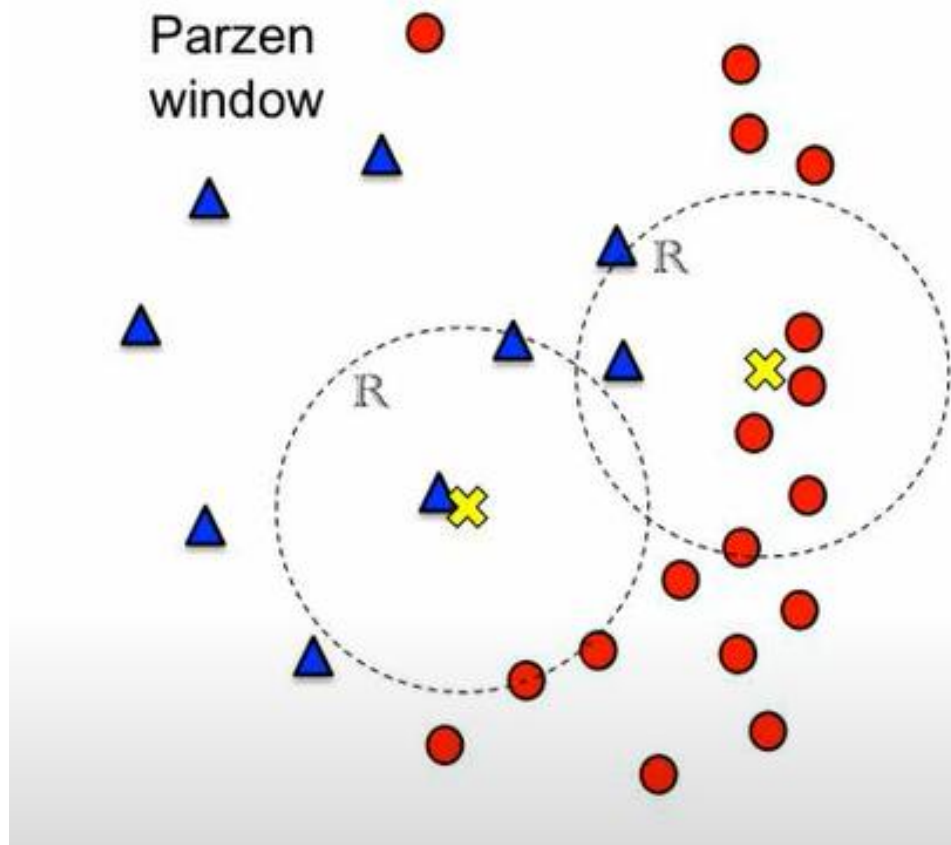
NONPARAMETRIC TECHNIQUES

- The general expression for **nonparametric density estimation** is

$$P(x) \cong \frac{k}{NV} \quad \text{where} \quad \begin{cases} V \text{ is the volume surrounding } x \\ N \text{ is the total number of examples} \\ k \text{ is the number of examples inside } V \end{cases}$$

- When applying this result to practical density estimation problems, two basic approaches can be adopted
 - Fix V and determine k from the data. This leads to **kernel density estimation (KDE)**, e.g. Parzen window.
 - Fix k and determine V from the data. This gives rise to the **k -nearest-neighbor (k -NN) approach**.
- Both k -NN and KDE converge to the true probability density as $N \rightarrow \infty$, provided that V shrinks with N , and that k grows with N appropriately.

KNN VS. PARZEN WINDOWS





PARZEN WINDOW

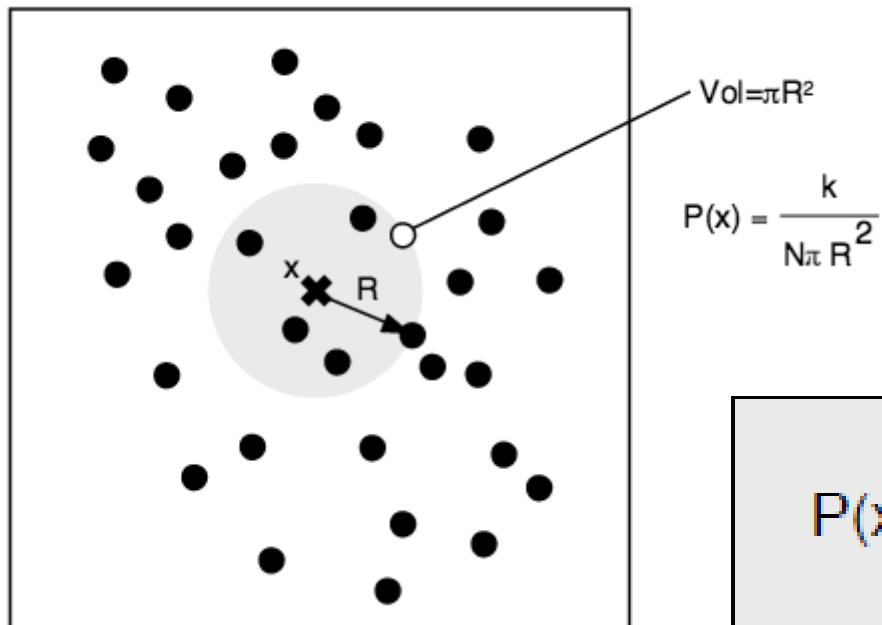
Parzen window is great for low-dimensional data (2–3 features, maybe up to 5).

In higher dimensions, the method becomes computationally expensive, and the density estimates are unreliable because of the curse of dimensionality.

For this reason, in A.A. 2025–2026, we focus on other methods better suited for high-dimensional data.

K-NEAREST NEIGHBORS (K-NN)

- The **volume** is grown surrounding the estimation point x , until it encloses a total of k data points.



$$P(x) \cong \frac{k}{NV} \quad \text{where} \quad \begin{cases} V \text{ is the volume surrounding } x \\ N \text{ is the total number of examples} \\ k \text{ is the number of examples inside } V \end{cases}$$

K-NEAREST NEIGHBORS (K-NN)

- Given an unknown point x^* , we consider $s_i \in \{s_1, \dots, s_N\} = S$ the nearest point (NN) of the point x^* if

$$d(x^*, s_i) = \min_l d(x^*, s_l), \quad l = 1 \dots N$$

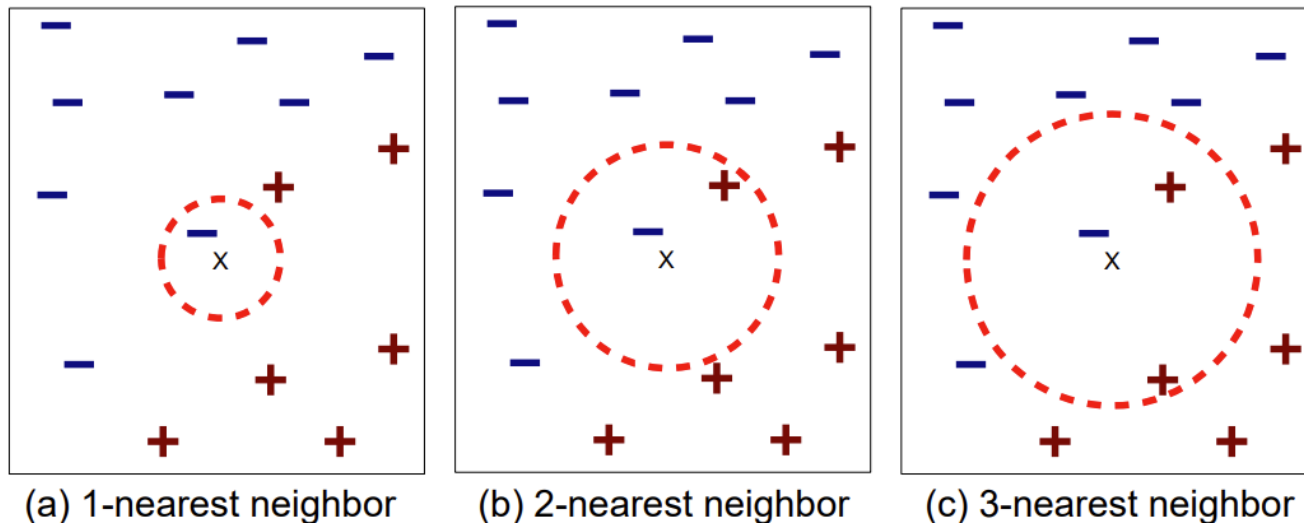
where $d(.)$ is a **distance measure** defined on the feature space.

- If you interpret each point of set S as a prototype of a class, you can then see the equation above as a rule of classification 1-NN associating the sample x to the class j to which the point s_i belongs.

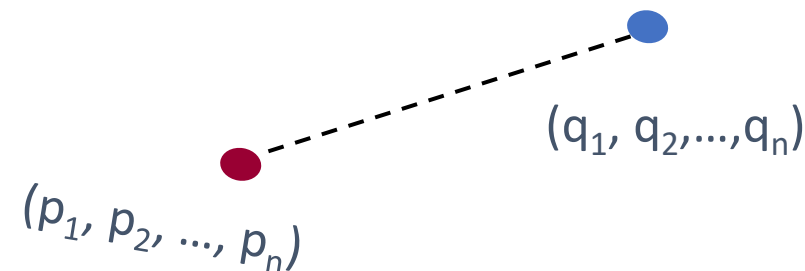
K-NEAREST NEIGHBORS (K-NN)

- Rule 1-NN can be generalised to define a k -NN rule, which consists of determining the k points belonging to the set S closest to the observation x .
- The classification rule k -NN associates observation x with class i having the largest number of elements among the nearest k .
- We call $U(x)$ the set of k points closer to x .
 - For example, with odd k and two classes, y_1 and y_2 , the decision rule can choose the class with the most samples in $U(x)$. ➔
majority voting

K-NEAREST NEIGHBORS (K-NN)



- To calculate the distance, we usually use **Euclidean distance**.



$$d(p, q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

K-NEAREST NEIGHBORS (K-NN)

- All the closest neighbors has the same weights, meaning that they are equally important to make a decision.
 - Therefore, the algorithm is very sensitive to the number of k .
- This can be reduced by **weighing** the contribution of neighbors based on distance $w = 1/d(x^*, s_i)$, such that the **closest** neighbor would have more **importance**.

K-NEAREST NEIGHBORS (K-NN)

- The choice of k is important because:
 - If k is too **small**, the approach is **sensitive** to **noise**
 - If k is too **large**, the neighborhood can include examples belonging to **other classes**
- To operate correctly the attributes must have the **same value scale** and must therefore be **normalized** in the preprocessing phase.

K-NEAREST NEIGHBORS (K-NN)

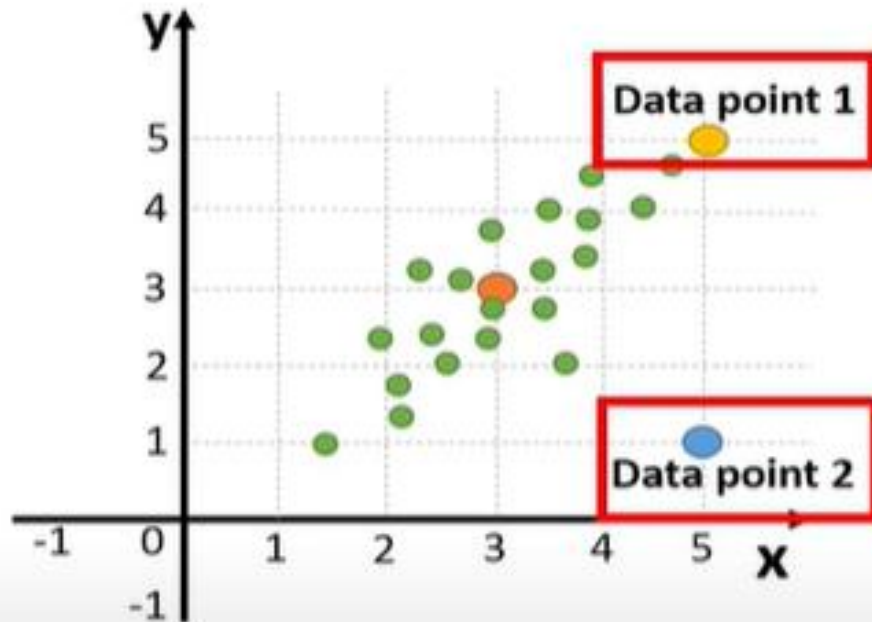
- Normalizing the scale may not be sufficient when there are different data distributions.
 - In such cases instead of using Euclidean distance, one can also try using Mahalanobis distance.
- The Mahalanobis distance is a measure of the distance between a point and a distribution. It's a generalized form of the Euclidean distance that accounts for correlations between variables and different variances along different axes.

$$D = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

- $\mathbf{x}$ is the n -dimensional vector representing the point's coordinates.
- $\boldsymbol{\mu}$ is the n -dimensional vector representing the mean of the distribution.
- $\boldsymbol{\Sigma}$ is the $n \times n$ covariance matrix of the distribution.
- $\boldsymbol{\Sigma}^{-1}$ is the inverse of the covariance matrix.

K-NEAREST NEIGHBORS (K-NN)

- When to use **Mahalanobis distance** instead of Euclidean distance?
 - **Correlated Features:** changes in one feature are associated with changes in another.
 - **Unequal Variances:** MD normalized the distance based on the variances of the features, preventing features with larger variances from dominating the distance calculation.
 - **Outlier Detection:** Outliers often have a large MD from the rest of the data points, especially if they deviate from the normal covariance pattern.
 - In **high dimensional spaces**, ED can be problematic (*curse of dimensionality*). Incorporating information about the covariance structure, can mitigate this issue to some extent.



$$\text{Centroid} = \begin{pmatrix} \bar{X} \\ \bar{Y} \end{pmatrix} = \begin{pmatrix} 3.1 \\ 3.0 \end{pmatrix}$$

Euclidean distance between the centroid and data point 1:

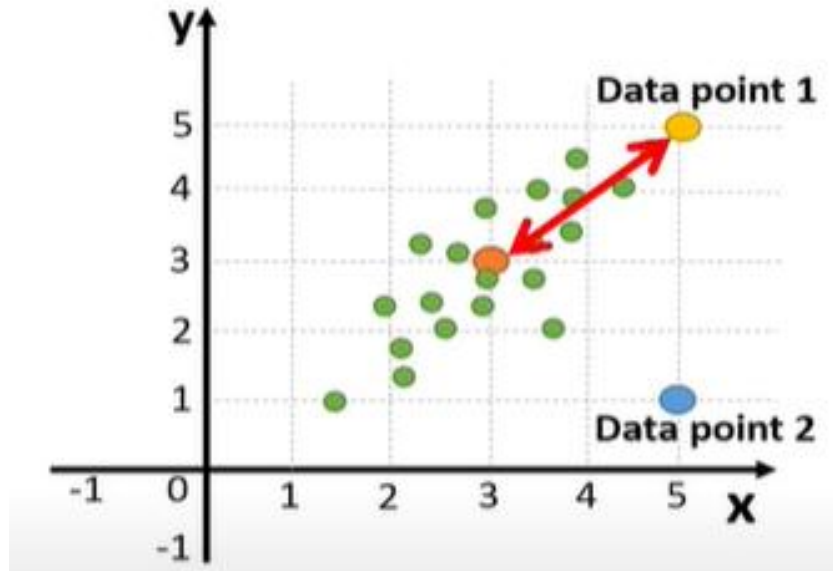
$$d = \sqrt{(5 - 3.1)^2 + (5 - 3.0)^2} = 2.76$$

Euclidean distance between the centroid and data point 2:

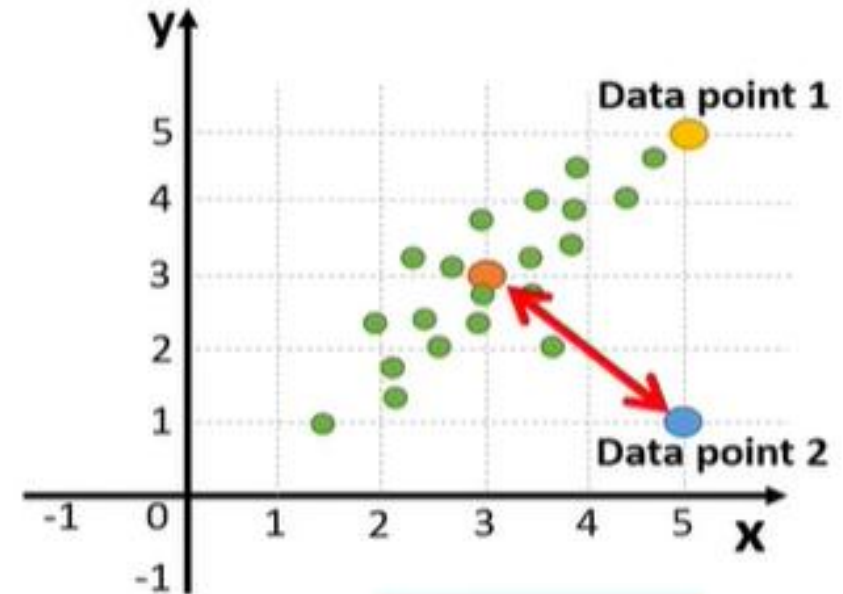
$$d = \sqrt{(5 - 3.1)^2 + (1 - 3.0)^2} = 2.76$$

- Based on Euclidean dist. both points have the same distance to the centroids

MAHALANOBIS DISTANCE



$$MD_1 = 2.26$$



$$MD_2 = 6.45$$

- Why MD1 is so different from MD2?

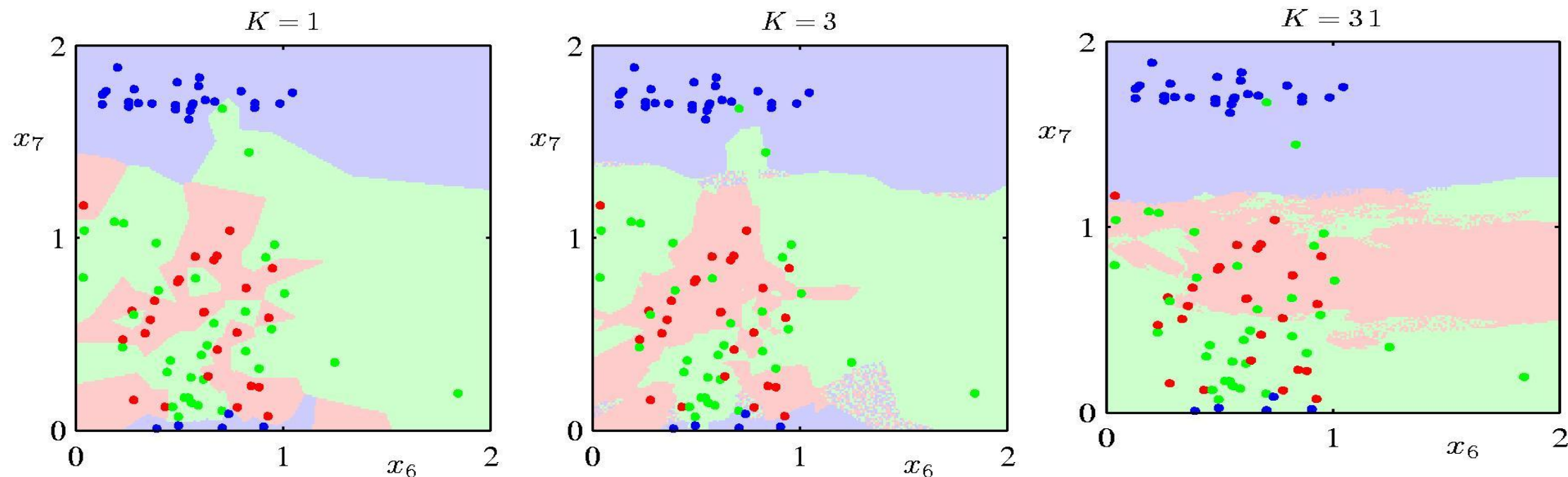
Bcs, MD takes into account the **correlation** in the data.

K-NEAREST NEIGHBORS (K-NN)

How to select ***k*** in k-NN?

- Common heuristics:
 - often 3, 5, 7
 - choose an **odd number** to avoid ties (*undetermined cases*)
- Use your training and validation data to decide ***k***.
 - Change ***k*** from ***k=1*** until the validation performance decreases.
 - Pay attention not to underfit and overfit the data.
 - Finalize the decision of ***k*** and use the same ***k*** to classify the test data.
- A typical choice is $k \cong \sqrt{N}$

HOW TO DECIDE K OF K -NN?



What is the role of k ?

How does it relate to **overfitting** and **underfitting**?

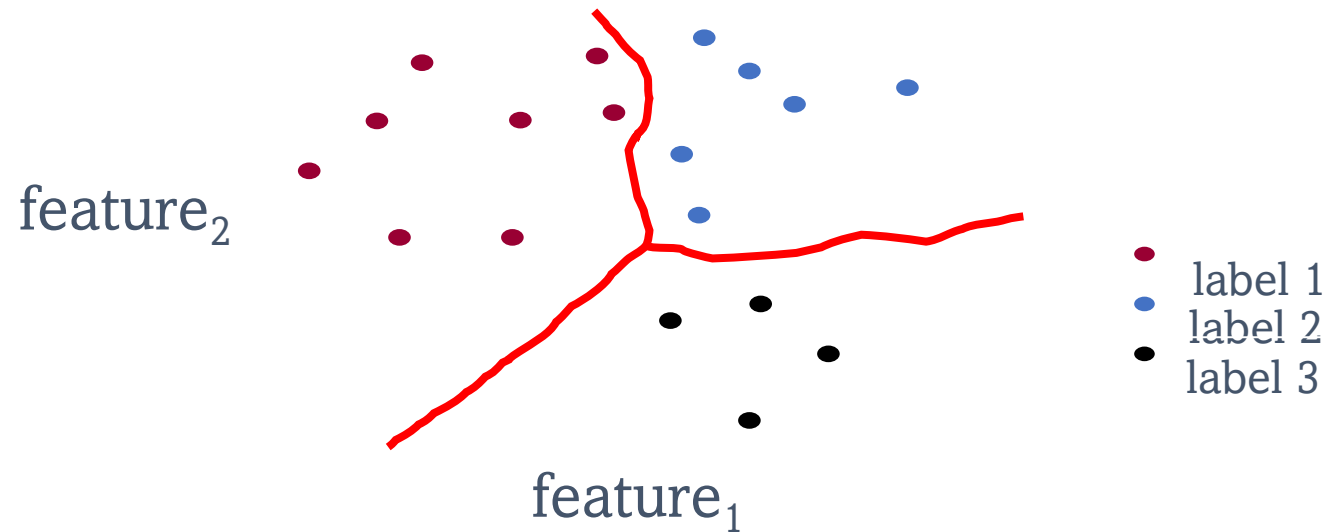
K-NEAREST NEIGHBORS (K-NN)

- If there are undetermined cases (ties) there are some alternatives:
 - Arbitrary choice
 - Assign x to the class (among those "tie") that has the nearest average sample (calculated between the k_i samples)
 - Assign x to the class that has the k_i samples with least distance from it

K-NEAREST NEIGHBORS (K-NN): PROS

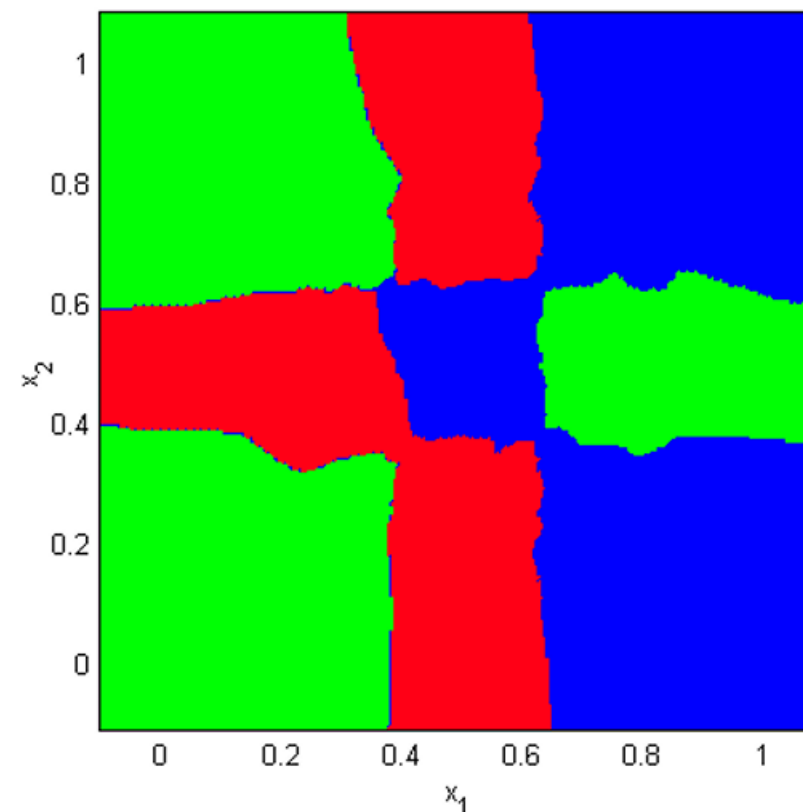
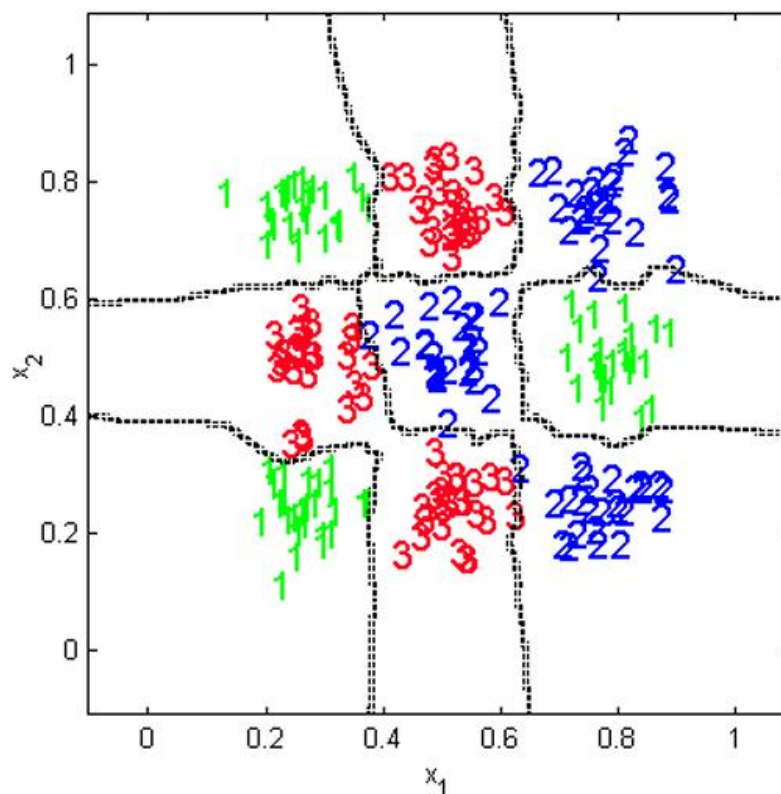
- Almost no assumptions about the data
- Non-parametric
- There is no training phase, cost is 0
- Allow us to construct **non-linear** class boundaries and are therefore more flexible.

k-NN gives **locally**
defined decision
boundaries between
classes



K-NEAREST NEIGHBORS (K-NN): PROS

- Allow us to construct **non-linear** class boundaries and are therefore more flexible.





K-NEAREST NEIGHBORS (K-NN): CONS

- Need to handle missing data.
- The distance metric should be chosen carefully.
- Sensitive to class outliers (e.g., mislabeled training instances)
- Sensitive to many irrelevant attributes (which influence the distance)
- Computationally expensive:
 - Poor scalability. For each new example, we have to perform as many comparisons as there are labeled data.
 - It is necessary to store all data points. Therefore, the method is more appropriate when the number of samples N is low.

HOW K-NN WAS BORN: PDF ESTIMATION



UNIVERSITÀ
di **VERONA**

Going back to the pdf estimation.....

- Instead of fixing V as in the case of kernels, we fix the value of k and enlarge the radius of the sphere to include k samples
- Let's see the **a-priori probability**:
 - Given a class y_j , the frequency of occurrence of samples N_j of class j is generally simply measured in relation to the total number of samples N , i.e.,

$$P(y_j) \equiv \hat{p}(y_j) = \frac{N_j}{N}$$

DENSITY ESTIMATION AND NEAREST-NEIGHBOR RULE



UNIVERSITÀ
di **VERONA**

- 2 classes, y_i and y_j , containing N_i and N_j samples, $N = N_i + N_j$
- The local density estimation for y_i is calculated as (and similarly for y_j)

$$\hat{p}(x|y_i) = \frac{1}{V} \frac{k_i}{N_i}$$

such that the ratio of k_i (k_j) points to the total N_i (N_j) belonging to the class y_i (y_j) contained in the volume V .

The Bayes rule says $p(x|y_i)P(y_i) > p(x|y_j)P(y_j)$, then

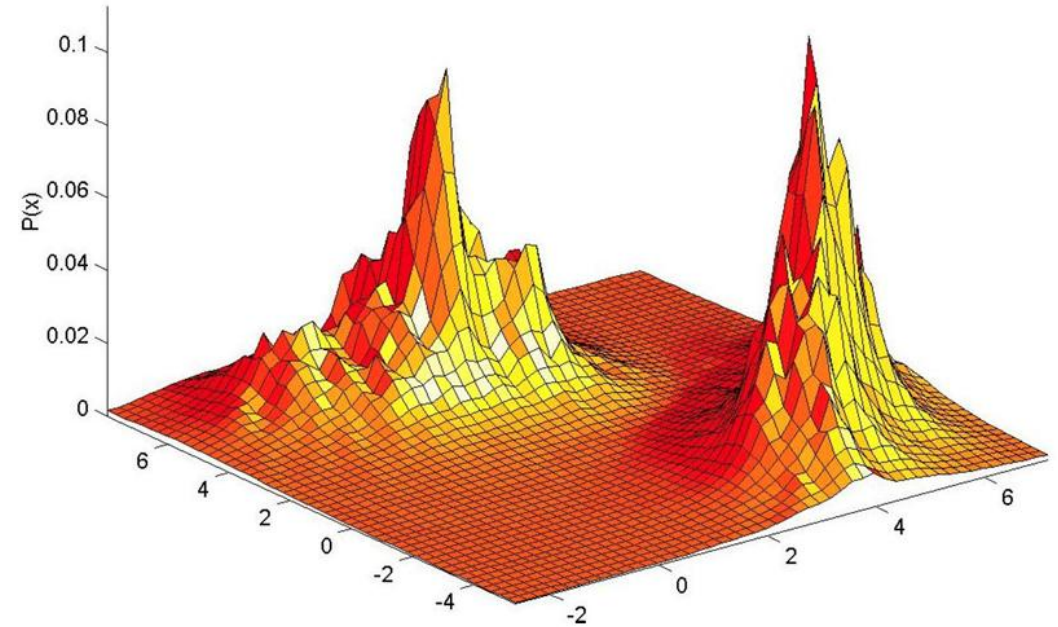
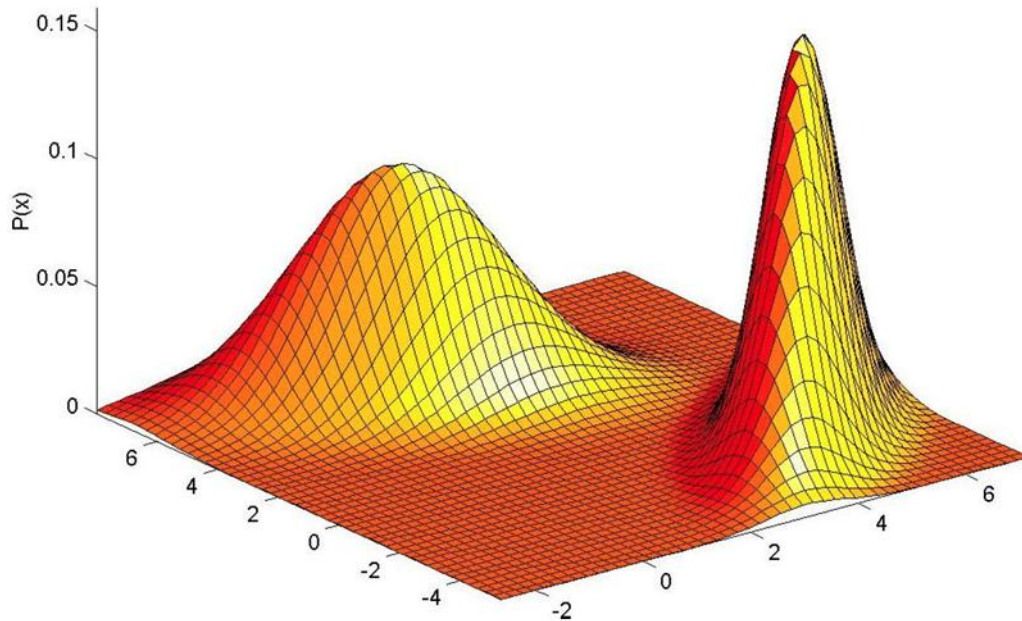
$$\begin{aligned} \hat{p}(x|y_i)\hat{p}(y_i) &> \hat{p}(x|y_j)\hat{p}(y_j) \\ \Rightarrow \frac{1}{V} \frac{k_i}{N_i} \frac{N_i}{N} &> \frac{1}{V} \frac{k_j}{N_j} \frac{N_j}{N} \quad \Rightarrow \quad k_i > k_j \end{aligned}$$

DENSITY ESTIMATION AND NEAREST-NEIGHBOR RULE



UNIVERSITÀ
di **VERONA**

The true density, a mixture of two bivariate Gaussians



Density estimate for $k=10$ neighbors
and $N=200$ examples.

- Notice that the estimate can be “jagged” and discontinuities in the slopes generally occur along lines away from the positions of the points themselves.



That's all

Cigdem Beyan
cigdem.beyan@univr.it
<https://cbeyan.github.io/>